# QMSP – A Chat Protocol over QUIC

## 1. Introduction

QUIC Message Sending Protocol (QMSP) is a chat protocol which allows the transfer of data across a network. The primary focus of this protocol is text-based data, but the system is extensible to allow the addition of more data types in the future. This is because message data is divided into various frame types similar to QUIC.

The protocol in version 1 is to support two text encoding modes, specifically UTF-8 and ASCII. However, the architecture in place to negotiate this is designed to allow legacy compatibility with other encoding modes, as well as to support a theoretical future encoding mode if UTF-8 were to be superseded as the standard.

## 2. Message Types

QMSP Messages can be broadly sorted into two categories: Data Messages and Control Messages. Data Messages only carry some form of data, such as plaintext or text formatting. Control Messages are primarily used to affect the conversation state, but most can contain data as part of their payload as well. This is to prevent an excess of control messages from holding up the server as well as to reduce roundtrip times needed for communication. For all messages, the first bit is the "Message Class," which is 0 for a Data Message and 1 for a Control Message.

### 2.1 Data Messages

There is only one specific type of Data Message. A data message consists of a header and a payload. The payload is subdivided into one or more frames. In version 1 of this protocol, this works to separate metadata for message verification into its own container for easy access. This also allows better extensibility to allow more datatypes in the future. The ability to transfer these within their own dedicated frame should be helpful for implementing such functionality.

```
Data Message {
  Message Class (1) = 0,
  Unused (2),
  Message Number Length (2),
  Chat ID Length (2),
  Unused (1),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  Post Payload (..)
```

*Figure 1: Layout of a Data Message*

Data Messages contain the following fields:

**Message Class:** The most significant bit of byte zero is set to 0 for a data message.

**Message Number Length:** A two-bit value dictating the number of bytes the Message Number integer is in length.

**Chat ID Length:** A two-bit value dictating the number of bytes the ChatID integer is in length.

**Chat ID:** A 1-4 byte value corresponding to the identification number of the target chat on the target machine.

**Source ID:** A variable length integer indicating the user who is attempting to message this chat.

**Message Number:** A 1-4 byte value marking the number of this message.

### 2.2 Control Messages

There are four main types of Control Messages. Connection Establishment messages help control connecting and disconnecting from different chats. User Management messages help alter information about a user, generally specific to a singular chat. Handshake messages are used to establish the secure connection between the client and the server which is hosting a given chat. Version Negotiation messages are used to negotiate the version of QMSP which the two endpoints will use to communicate with each other. As this is version 1 of the protocol, these are unlikely to be particularly needed, but it's anticipated that these will be useful in the future as the protocol evolves.

The type of control message is defined by the two bits following the first in the header byte, such that 0 identifies Connection Establishment, 1 identifies User Management, 2 identifies Handshake, and 3 identifies Version Negotiation. These two bits are unused in Data Messages.

```
Control Message {
  Message Class (1) = 1,
  Control Type (2),
  Message Number Length (2),
  Chat ID Length (2),
  Type-Specific Bit (1),
  Version Number (32),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  Type-Specific Payload (..)
}
```

*Figure 2: Layout of a general Control Message*

Control Messages contain the following fields:

**Message Class:** The most significant bit of byte zero is set to 1 for a Control message.

**Control Type:** A 2-bit value dictating the type of control message it is.

**Message Number Length:** A two-bit value dictating the number of bytes the Message Number integer is in length.

**Chat ID Length:** A two-bit value dictating the number of bytes the ChatID integer is in length.

**Type Specific Bit:** The behavior of this flag bit is dependent on control message type.

**Version Number:** The version number being used for this communication.

**Chat ID:** A 1-4 byte value corresponding to the identification number of the target chat on the target machine.

**Source ID:** A variable length integer indicating the user who is attempting to message this chat.

**Message Number:** A 1-4 byte value marking the number of this message.

**Type Specific Payload:** The remainder of the message is dependent on the control message type.

### 2.2.1 Version Negotiation Messages

Version Negotiation Messages are the most different from the others in terms of PDU structure. Firstly, they are only type which do not have the functionality to carry a post payload. Version Negotiation Packets cannot have such functionality, as they exist to negotiate the version to use, meaning the other endpoint may not be able to understand the message data. Version Negotiation messages are generally invalid after connection is established, as the version would have already been negotiated, and developers SHOULD NOT let them be sent after the connection is established. As a security measure, developers MAY implement detection for such behavior and terminate connections with endpoints which attempt to do this.

If the initial version number does not match the highest supported number at the receiving end point, the recipient SHOULD send a version negotiation packet to the sender. This SHOULD be responded to with another version negotiation packet containing the highest common version number as the first entry of its message data. This version number SHOULD be assigned as the version to be used for this connection. Developers SHOULD treat this agreement as implicit and have both endpoints set this without further negotiation. If, for whatever reason, a developer desires a more explicit agreement, they MAY otherwise implement such by exchanging two more Version Negotiation packets using the desired version as the only entry.

```
Version Negotiation Message {
  Message Class (1) = 1,
  Control Type (2) = 3,
  Message Number Length (2),
  Chat ID Length (2),
  Unused (1),
  Version Number (32),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  Supported Versions (..)
}
```

*Figure 3: Layout of a Version Negotiation Message*

In addition to values common in all Control Messages, Version Negotiation Messages have the following fields:

**Supported Versions:** A list of supported versions of the QMSP protocol, listed end-to-end in four bytes per entry.

### 2.2.2 Connection Establishment Messages

Connection Establishment Messages are primarily used to establish the connection to a specific chat on the target machine.

```
Connection Establishment Message {
  Message Class (1) = 1,
  Control Type (2) = 0,
  Message Number Length (2),
  Chat ID Length (2),
  Data Flag (1),
  Version Number (32),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  Connection Establishment Control Frames (..),
  [Post Payload (0..)]
}
```

*Figure 4: Layout of a Connection Establishment Message*

In addition to values common in all Control Messages, Connection Establishment Messages have the following fields:

**Data Flag:** A singular bit which marks whether this message contains a Post Payload equivalent to that of a Data Message.

**Connection Establishment Control Frames:** A set of frames which are exclusive to Connection Establishment Messages that facilitate the connection to a chat.

**Post Payload:** Equivalent to that of a Data Message.

### 2.2.3 User Management Messages

User Management Messages are primarily used to alter data about a user on a specific chat.

```
User Management Message {
  Message Class (1) = 1,
  Control Type (2) = 1,
  Message Number Length (2),
  Chat ID Length (2),
  Data Flag (1),
  Version Number (32),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  User Management Control Frames (..),
  [Post Payload (0..)]
}
```

*Figure 5: Layout of a User Management Message*

In addition to values common in all Control Messages, User Management Messages have the following fields:

**Data Flag:** A singular bit which marks whether this message contains a Post Payload equivalent to that of a Data Message.

**User Management Control Frames:** A set of frames which are exclusive to User Management Messages that facilitate altering user information.

**Post Payload:** Equivalent to that of a Data Message.

### 2.2.4 Handshake Messages

Handshake Messages are primarily used to negotiate options and conditions for the connection.

```
Handshake Message {
  Message Class (1) = 1,
  Control Type (2) = 2,
  Message Number Length (2),
  Chat ID Length (2),
  Data Flag (1),
  Version Number (32),
  Chat ID (8..32),
  Source ID (i),
  Message Number (8..32),
  Handshake Control Frames (..),
  [Post Payload (0..)]
}
```

In addition to values common in all Control Messages, Handshake Messages have the following fields:

**Data Flag:** A singular bit which marks whether this message contains a Post Payload equivalent to that of a Data Message.

**Handshake Control Frames:** A set of frames which are exclusive to Handshake Messages that facilitate option negotiation.

**Post Payload:** Equivalent to that of a Data Message.

## 3. Variable Length Integer Encoding

Several values in the packet and frame layout utilize variable length integer encoding much like is seen in QUIC, allowing smaller values to be encoded in fewer bits. The system works exactly the same as in QUIC, such that the two most significant bits encode the length of the integer as a base-2 logarithm. The remainder of the bits encode the integer value. The variable “i” is typically used to display such a variable length integer in figures and descriptions throughout this document.

## 4. Payload Delineation

The payload present in a Message is subdivided into distinct segments known as frames. Each frame has a distinct data layout, and each one can define a distinct type of data. A single frame is at least a single byte in length, with the first byte always containing the identifier for what type of frame it is. Beyond this, the size of a single frame is variable and depends on the frame type and its internal definitions. All frames are ordered with the most significant byte at the lowest memory location value.

```
Frame {
  Type (i),
  Frame-specific Data (..)
}
```

*Figure 6: Layout of a frame*

The type of a frame is a variable length integer, allowing new types of frames to be defined easily for future extensions.

## 5. Data Frames

Data frames are part of the post payload for Data Messages and eligible Control Messages. They carry the data for a post within the chat. Every message with a post payload contains at least one Data frame.

### 5.1 Transit Metadata

Every message with a post payload MUST contain a Transit Metadata frame.

A Transit Metadata frame is identified by the byte value 0x00 and contains metadata about the message being transmitted which is important to ensure it is transferred properly.

```
Transit Metadata Frame {
  Type (i) = 0x00,
  PostTransitID (32),
  TotalMessageCount (i),
  SpecificMessageCount (i)
}
```

*Figure 7: Layout of a Transit Metadata Frame*

Transit metadata contains the following fields:

**PostTransitID:** An integer value which should be unique per post for the current connection. This value is intended to ensure that any post which must be split between multiple messages can be reassembled fully at the opposite end.

**TotalMessageCount:** An integer value which identifies how many messages the post is split across. This is important for ensuring that all pieces of the message have been received.

**SpecificMessageCount:** An integer value which identifies where in the sequence of messages this segment of the post resides. This allows us to easily identify any segments which may have been lost in transit.

### 5.2 Post Metadata

A Post Metadata frame is identified by the byte value 0x01 and carries data regarding specific characteristics of the post.

```
Post Metadata Frame {
  Type (i) = 0x01,
  PostID (i),
  Timestamp (64),
  [Linkage {
    LinkType (8),
    LinkID (i)
  }],
  0x00
}
```

*Figure 8: Layout of a Post Metadata Frame*

Post Metadata frames contain the following fields:

**PostID:** An integer value unique to the current connection. This is primarily used to reference previously loaded messages in the current connection.

**Timestamp:** An eight byte value storing a 64-bit unix time value corresponding to the time at which the post was sent. This is used to keep past posts ordered chronologically.

**Linkage:** A data structure used to link a given post to other data, primarily users or other posts. This section may be omitted if there are no links required. The data consists of several byte sequences, where the first byte in said sequence dictates the type of link and the remainder of the sequence encodes said link. There can be any number of links listed.

**LinkType:** A byte used within the Linkage list at the start of a new sequence. Presently, the only link types present are 0x01 for a user link and 0x02 for a post link. There is room to add more in the future, with the only limitation being that no link type can be 0x00, as this is used to mark the end of the Linkage list.

**LinkID:** A variable length integer encoding either a UserID or a PostID depending on the LinkType byte preceding it. In either case, it is a variable length integer. If the value is a PostID, it is a value which is unique to the current connection, not necessarily an id stored in a server database.

### 5.3 Plaintext

A plaintext frame is identified by the byte values 0x02 and 0x03 and contains plaintext. Plaintext frames simply transmit raw, unformatted textual data. These are ideal to send when no text formatting is needed.

For the type byte of a plaintext frame, the presence of a length field is determined by the value. A length field is present for type 0x03 and absent for type 0x02.

```
Plaintext Frame {
  Type (i) = 0x02..0x03,
  [Length (i)],
  Text Data (..)
}
```

*Figure 9: Layout of a Plaintext Frame*

Plaintext frames contain the following fields:

**Length:** A variable length integer dictating the length of the textual data following it. It is present in plaintext frames of type 0x03. In cases where this is absent, the remainder of the message is treated as textual data within this plaintext frame.

**Text Data:** A Long string of textual data. The data can be encoded in either UTF-8 or ASCII depending on the encoding negotiated during the handshake.

### 5.4 FText

An FText frame is identified by the byte values 0x04 and 0x05 and contains text with formatting parameters. For all text which requires formatting, it is surrounded by special characters. As the backspace character in both UTF-8 and ASCII doesn't particularly make sense to appear in the text of posts, we can utilize the corresponding byte value in the protocol itself to serve as a flag for the beginning and end of a formatted segment. This character is the byte value 0x08. Additionally, the first byte within the segment should contain a formatting value, which corresponds to one of the entries in a Formatting Frame, known as a formatting index. This will explain what formatting is to be applied to this segment. Segments that are to have the same formatting can use the same formatting index.

For the type byte of an FText frame, the presence of a length field is determined by the value. A length field is present for type 0x04 and absent for type 0x05.

```
FText Frame {
  Type (i) = 0x04..0x05,
  [Length (i)],
  Text Data (..){
    [Text (..)],
    [Formatted Segment {
      0x08,
      FormattingIndex (8),
      Text (..),
      0x08
    }]
  }
}
```

*Figure 10: Layout of an FText Frame*

FText frames contain the following fields:

**Length:** A variable length integer dictating the length of the textual data following it. It is present in FText frames of type 0x05. In cases where this is absent, the remainder of the message is treated as textual data within this FText frame.

**Text Data:** A Long string of textual data. The data can be encoded in either UTF-8 or ASCII depending on the encoding negotiated during the handshake. In either case, 0x08 bytes are used as flags to indicate the start and end of a formatted segment. This corresponds to the backspace character (\b), which doesn't particularly make sense to be present within a message.

**FormattingIndex:** A single byte indicating the formatting style to apply to the given segment of the Text Data.

### 5.5 Formatting

A Formatting frame is identified by the byte value 0x06 and contains sequences of formatting parameters to be applied to text within an FText frame.

```
Format Frame {
  Type (i) = 0x06,
  IndexCount (8),
  Formatting Index List {
    Index (8),
    SimpleFormat (8),
    [FontSize (8)],
    [Color (24)]
  }
}
```

*Figure 11: Structure of a Formatting Frame*

Format frames contain the following fields:

**IndexCount:** The number of indices present in the Frame's Formatting Index List. There can be a maximum of 256 distinct formatting styles for different segments.

**Formatting Index List:** A variably sized list of indices and associated formatting styles. This contains a number of byte sequences equal to the index count. The following are the facets of these byte sequences.

**Index:** The index associated with the formatting data which follows it.

**SimpleFormat:** A byte controlling both the size of the formatting data and specific simple text effects. The first two bits dictate the presence of FontSize and Color data bytes. The remaining six bits are flags as to whether a specific text effect should be applied. These are, in order from most significant to least significant bit: Bold, Italic, Underline, Strikethrough, Subscript, and Superscript. Table 1 lays this out visually.

| Include Color | Include Size | Bold | Italic | Underline | Strike-through | Subscript | Super-script |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

*Table 1: Layout of the SimpleFormat Byte in a Format Frame*

**FontSize:** A byte which is present if the second bit of the SimpleFormat byte is set to 1. This controls the font size, with its value directly translating to the font size in points. The maximum font size is thus 256 point.

**Color:** Three bytes which are present if the first bit of the SimpleFormat byte is set to 1. This controls font color, with the three bytes directly translating to RGB color values in that order.

### 5.6 Load

A Load Frame is identified by the byte value 0x07 and is used to request previous posts in the chat history.

```
Load Frame {
  Type (i) = 0x07,
  PostID (i),
  Timestamp (64)
}
```

*Figure 12: Layout of a Load Frame*

Post Metadata frames contain the following fields:

**PostID:** An integer value unique to the current connection of the last post received. This is primarily used to reference what the previously loaded message was in order to fetch the next ones from the chat history.

**Timestamp:** An eight byte value storing a 64-bit unix time value corresponding to the time at which the last post received was sent. This is used as a fallback in case the post id is incorrect for some reason.

## 6. Control Frames

A subset of the defined frames is only valid to be used in specific Control Messages. Developers MUST ensure that no messages are able to send invalid frames for its message type. These Control Frames are not considered part of the post payload, as these frames are all related to control functions rather than textual data for posts.

### 6.1 Encoding Negotiation

Encoding Negotiation Frames are only valid in Handshake Messages.

They are identified by the byte value 0x08 and contain data for setting up the encoding mode of text in this connection. The first encoding mode supported by both endpoints should be selected as that used for this connection.

```
Encoding Negotiation Frame {
  Type (i) = 0x08,
  EncodingSupport (8)
}
```

*Figure 13: Structure of an Encoding Negotiation Frame*

Encoding Negotiation frames contain the following fields:

**Encoding Support:** A byte where each value is treated as a flag that a specific encoding mode is supported. The preferred encoding modes are such that the most preferred encoding mode is in the least significant bit and the least preferred encoding mode is in the most significant bit.

The encoding mode is chosen by bit-wise and-ing the two endpoints' support values and picking the one corresponding to flag of value 1 in the least significant position.

In version 1, only two encoding modes are supported in ASCII and UTF-8, which are laid out such that UTF-8 Is the preferred default. This means that the first six bits are unused, but they may be used to implement support for other legacy encoding modes in the future. The current mapping is shown in figure 7.

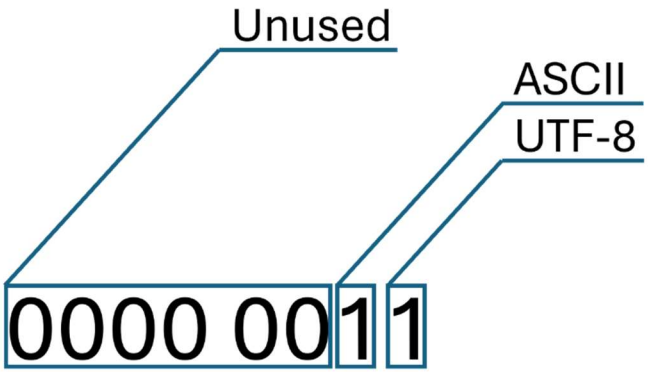


*Figure 14: Byte Layout of the EncodingSupport*

The reason the least significant position is used for this is to future-proof the design. If at some point in the future UTF-8 were to be superseded as the standard encoding mode, we could add a second byte to the encoding negotiation packet and set the new encoding mode to be the most significant bit of this. This is displayed visually in figure 8. This ensures that the first byte always lines up with those of older versions, simplifying the coding needed to support this change.

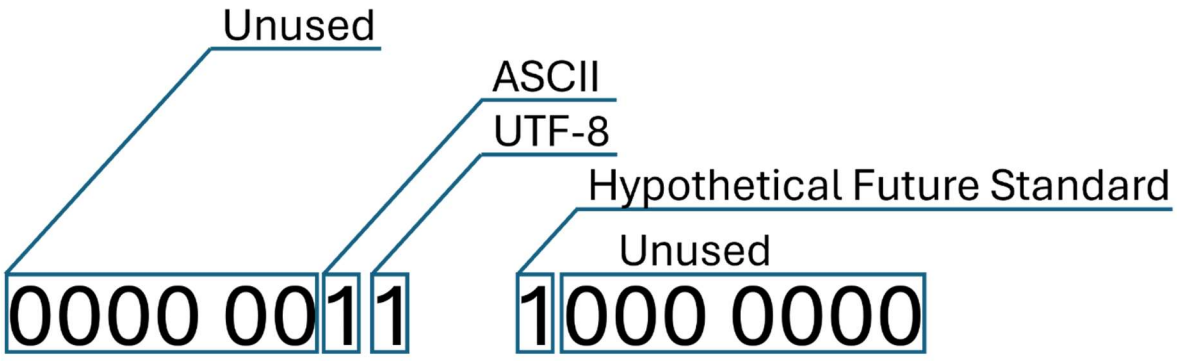


*Figure 15: Potential expansion of the EncodingSupport to default to a theoretical future standard.*

### 6.2 User Datagram

User Datagram Frames are only valid in User Management Messages.

They are identified by the byte value 0x09 and contain datagrams used to request changes to specific user information. This information is intended to apply to a specific chat or channel to which they are related.

```
User Datagram Frame {
  Type (i) = 0x09,
  Length (8),
  Datagram {
    Aspect (i),
    Data (..)
  }
}
```

*Figure 16: Structure of a User Datagram Frame*

User Datagram frames contain the following fields:

**Length:** A byte dictating the number of aspects present within the Datagram. A single User Datagram frame can request alterations to up to 256 aspects at once.

**Datagram:** A list containing different aspects of the user which alterations are being requested for. These take the form of a byte followed by a sequence of bytes with the new data.

**Aspect:** A variable-length integer where each value corresponds to a different aspect of the user on the given channel. Additional aspects may be added in the future.

**Data:** The new data to be assigned to the given aspect. The size and format is dependent on the aspect in question. Table 2 demonstrates the currently supported aspects and the data layouts.

| Integer | Aspect | Data Layout | Details |
|---|---|---|---|
| 1 | Nickname | `Length (8),`<br>`String Data (..)` | String Data has the number of bytes specified by the length. |
| 2 | Permission | `Permissions (8)` | Dictates what the user is allowed to do in this chat. Byte acts as a series of flags. This is laid out in Table 3. |
| 3 | Ban | `TimeSpan (8)` | Sets all permissions to 0 for a given time. Timespan is divided into two four-bit nybbles, where the first dictates units and the second dictates the amount. This is laid out in Table 4. A value of 0xff corresponds to a permanent ban. A value of 0x00 corresponds to lifting a ban |

*Table 2: Data Layout for each currently supported aspect*

| Change User Permissions | Change User Data | Change Own Data | Unused | Unused | Post | Read | Join |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |

*Table 3: Permissions Byte Layout*

| **Units** | | | | **Amount** | | | |
|---|---|---|---|---|---|---|---|
| Months | Weeks | Days | Hours | Length | | | |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

*Table 4: Ban Timespan Byte Layout*

### 6.3 Change Denial

Change Denial Frames are only valid in User Mannagement Messages.

They are identified by the byte value of 0x0A and are used to inform the sender that part of the changes specified in a previous User Datagram frame failed to go through. This is often due to such a change being restricted on a given chat. An example of why this might be used would be if a specific chat restricts users from changing their nickname and requires an administrator to perform this instead. Another example would be that the user is not connected to the chat specified by the Message.

```
Change Denial Frame {
  Type (i) = 0x0A,
  Permissions (8),
  Length (8),
  Denied Changes {
    Aspect (i)
  }
}
```

*Figure 17: Layout of a Change Denial Frame*

Change Denial frames contain the following fields:

**Permissions:** A byte dictating the user permissions in this channel to explain why the changes were denied.

**Length:** A byte dictating the number of aspects present within the Denied Changes. A single frame can deny up to 256 changes at once.

**Denied Changes:** A series of aspects which were requested to be changed that were denied.

**Aspect:** A variable length integer corresponding to different aspects of a user related to the channel. These can be seen in Table 2.

### 6.4 Change Approval

Change Approval Frames are only valid in User Mannagement Messages.

They are identified by the byte value of 0x0B and are used to inform the sender that the changes specified in a previous User Datagram frame successfully went through.

```
Change Approval Frame {
  Type (i) = 0x0B,
  Length (8),
  Approved Changes {
    Aspect (i)
  }
}
```

*Figure 18: Layout of a Change Approval Frame*

Change Approval frames contain the following fields:

**Length:** A byte dictating the number of aspects present within the Approved Changes. A single frame can approve up to 256 changes at once.

**Approved Changes:** A series of aspects which were requested to be changed that were approved.

**Aspect:** A variable length integer corresponding to different aspects of a user related to the channel. These can be seen in Table 2.

### 6.5 Login

Login Frames are only valid in Connection Establishment messages.

These are identified by the byte value 0x0C and are used to carry data to verify a user attempting to connect to a chat.

```
Login Frame {
  Type (i) = 0x0C
}
```

*Figure 19: Layout of a Login Frame*

For the most part, all of the information to verify a user is already present in the rest of the message. This frame primarily acts as a signal to try signing the user into a chat.

### 6.6 Connection Acknowledgement

Connection Acknowledgement Frames are only valid in Connection Establishment messages.

These are identified by the byte value 0x0D and are used to carry data regarding connections to different chats and channels. Developers SHOULD send two distinct Connection Acknowledgements for each chat connection. The first acknowledges that the request was received, while the second should verify whether the connection was successful or failed.

```
Connection Acknowledgement Frame {
  Type (i) = 0x0D,
  ConnectionStatus (8)
}
```

*Figure 20: Layout of a Connection Acknowledgement Frame*

Connection Acknowledgement frames contain the following fields:

**ConnectionStatus:** A byte relaying the result of attempting to connect to a given chat. The byte values corresponding to this are listed in table 5.

| Status Byte | Connection Status |
|---|---|
| 0000 0000 | Still Connecting |
| 0000 0001 | Connected |
| 0000 0010 | Connection Refused (Error) |
| 0000 0011 | Connection Refused (User is banned) |
| 0000 0100 | Connection Timeout |
| 1111 1111 | Error, Chat does not exist |

*Table 5: Connection Status and Corresponding byte values*

# 7. Extensibility

This protocol is designed to be extensible. The frame system allows much easier addition of new datatypes in the future. Additionally, data is laid out to try ensuring new additions, particularly if data ever needs to be expanded, do not break parity with old versions wherever possible, thus simplifying the code changes needed to do so. If more options are added negotiation can be implemented via new handshake control frames.

# 8. Security Considerations

As with any network protocol, several security considerations must be made. In terms of securing message data from unauthorized viewers, it would be ideal to ensure the messages are encrypted. Fortunately, QUIC encrypts the data in its packets by default via a TLS 1.3 handshake, meaning we get this for free.

Beyond this, the primary goal is to limit access to chats which need privacy. It is for this reason that the permissions byte exists. A user for whom the Join bit is set to 0 will not be allowed to connect to a given Chat. The chat owner can set up the default permissions for users to ensure that unauthorized users always default to 0. This bit also ensures that proper permissions are in place to prevent random users from changing crucial management data.

# 9. Deterministic Finite Automaton

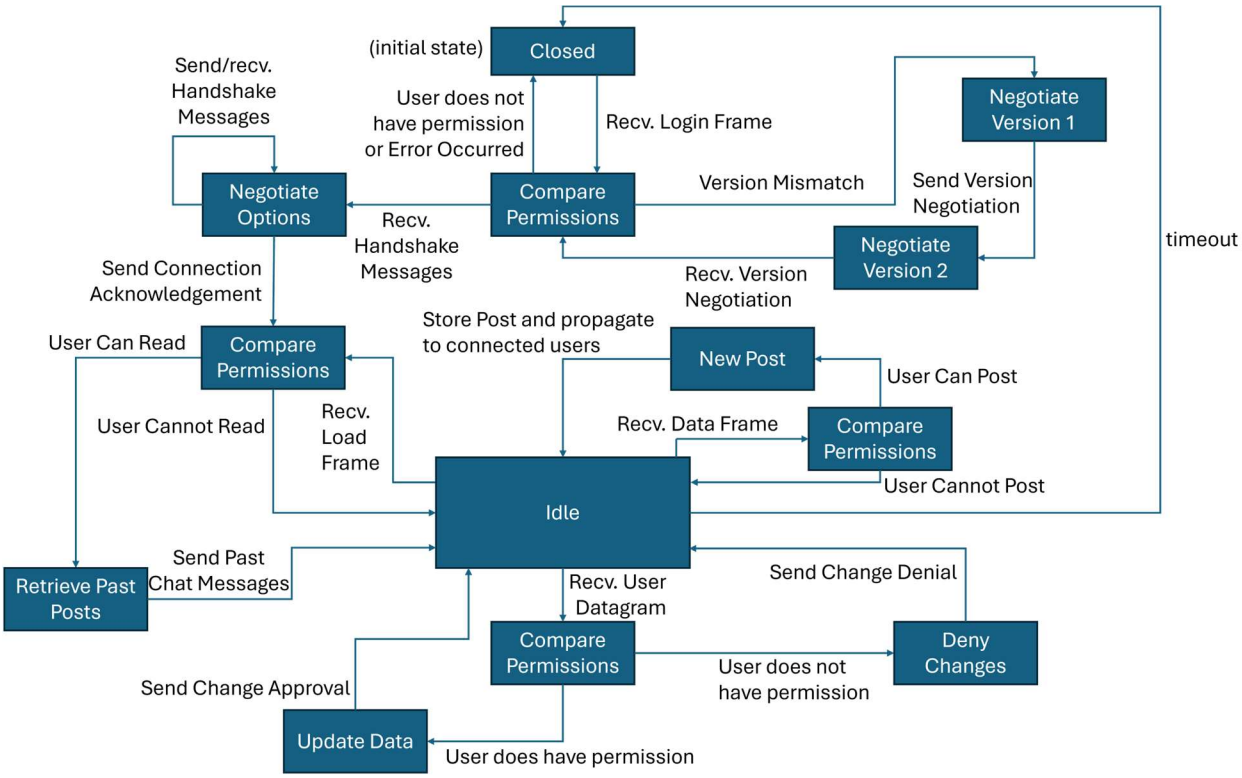


*Figure 21: DFA for the QMSP protocol*